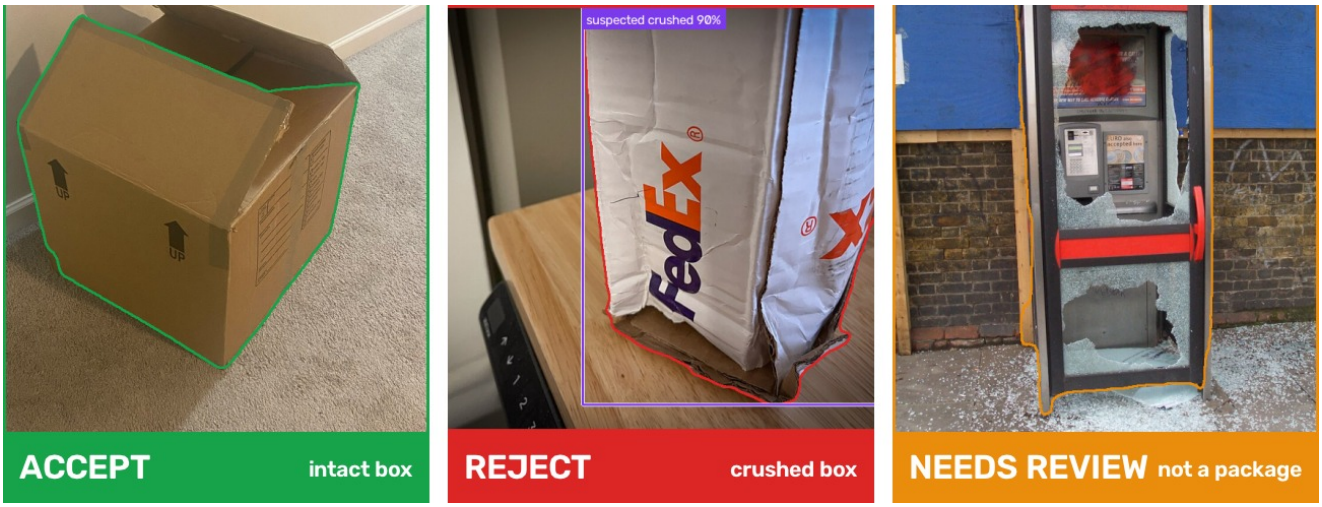
AI Package Damage Inspector

Project overview: what it does, how it works, and how well it works

The system inspects a photo of a parcel and makes a quality-control decision, the way a person at a warehouse dock would. For every photo it reports the decision, the type of damage, where the damage is, how severe it is and **why** it decided that.

ACCEPT package is fine **REJECT** crushed, torn or stained **NEEDS REVIEW** unsure or bad photo: a person checks



The three possible outcomes, produced by the app on real photos. The outline colour shows the decision; the violet box marks where the model located the crushing.



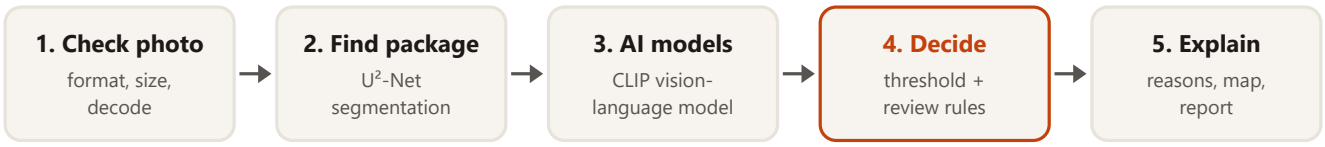
Inside one inspection (crushed FedEx box). Left to right: photo · package found by segmentation · damage-probability map · final result. Decision: **REJECT**, *damage type* **crushed**, *severity* **Severe**, *P(intact) = 28%, processed in 1.0 s.*

What it can do

- Inspect **sample photos, uploads, camera shots** or computer-generated packages.
- Classify packages as **intact, crushed, torn or water-stained**, and detect photos with **no package**.
- Show **where** the package is (segmentation) and **where** the damage is (damage map and boxes).
- Send **uncertain cases and bad photos** (blurry, dark, tiny) to a person instead of guessing.
- **Batch inspection**, a **conveyor-belt simulation**, a **permanent inspection history**, downloadable **annotated images and JSON reports**, and a live **model evaluation** page.

How it works

Every photo passes through the same four steps, whether it comes from a single upload, a batch or the conveyor simulation:



Two AI engines

	Real photos	Synthetic (computer-generated) packages
Model	**CLIP ViT-B/32** (pre-trained on 2 billion image-text pairs) used **zero-shot**: the photo is compared with descriptions such as "a crushed cardboard box"	**Random Forest** on 28 classical image features + a small **CNN**, averaged
Segmentation	**U²-Net** salient-object network	Otsu threshold + morphology
Damage location	CLIP scores 49 small windows → damage map; convexity defects mark dents	Rules: dark blobs (tears), brightness drop (stains), convexity defects (crushing)
Why	Works on real photos without a labelled dataset of damaged parcels	Shows the classical IVP pipeline; trained on generated images

How the decision is made

The models only output probabilities. A separate decision layer turns them into a business decision. Each check is tried in order; the first one that applies wins:



The **threshold** is a policy setting in the app (stricter = more rejections). Model **confidence**, the **threshold** and damage **severity** are kept separate: severity measures how much of the package is damaged, and is reported as "not assessed" if the damage could not be located.

Results

All numbers below were computed by running the full system (models + decision rules, threshold 0.65) on labelled images.

Synthetic test set: 200 new generated packages the models never saw during training

99.5%	0.7%	0.0%	2.5%
accuracy on decided packages	damaged packages accepted	intact packages rejected	sent to manual review

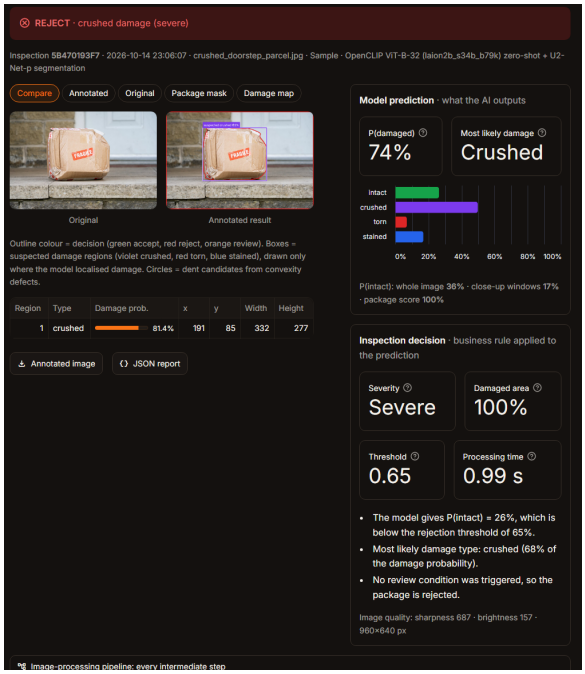
Real photos (11 bundled sample photos)

9/9	100.0%	10.0%	0.0%
decided packages correct	non-package photos caught	sent to review (low-res photo)	old synthetic-trained model on the same photos

Important: the real-photo thresholds were tuned on these same photos, so this is **not** an independent test. The last tile shows why the CLIP engine was needed: the models trained on generated boxes do not work on real photos (a "domain shift").

Example decisions

Photo	Decision	Type / severity	Why
crushed fedex box	**REJECT**	crushed / Severe	The model gives P(intact) = 28%, which is below the rejection threshold of 65%.
intact closed box	**ACCEPT**	- / -	The model gives P(intact) = 78%, which meets the rejection threshold of 65%.
not a package phone booth	**NEEDS REVIEW**	- / -	No package detected: package score 0% (needs 70%). Retake the photo with the parcel clearly visible.
stained archive box	**REJECT**	stained / Severe	The model gives P(intact) = 49%, which is below the rejection threshold of 65%.
intact packed box	**NEEDS REVIEW**	- / -	Resolution too low (190x127 px, minimum 160 px on the short side).



The main inspection screen: decision banner, original vs. annotated image, the model prediction card and the inspection decision card with its explanation.

Using the app

Page	Purpose
Inspect	One photo: decision, prediction, explanation, image views, downloads, processing pipeline
Batch inspection	Many photos at once, failed files listed, CSV / ZIP / JSON export
Conveyor simulation	Packages enter → are inspected → routed to accept, reject or review lanes (software simulation, no hardware)
Inspection history	Every inspection saved permanently; filter, search, open, re-download
Model evaluation	Accuracy, precision, recall, F1, confusion matrix, false accept / reject rates
How it works	Architecture, decision rules and limitations

Run it

`setup.bat` (first time: installs packages and downloads the CLIP model, ~600 MB), then `run_app.bat` and open `http://localhost:8501`.

Technology

Python · OpenCV · PyTorch · OpenCLIP · U^2-Net (ONNX, run with OpenCV DNN) · scikit-learn · SQLite · Streamlit

Limitations

- No large labelled dataset of real damaged parcels was available, so real-photo accuracy has not been measured on an independent test set (the app shows how to add one).
- CLIP is a general model: subtle or hidden damage is hard, and such cases often go to manual review.
- The damage map is coarse (window-based): it marks areas, not exact pixels.
- The conveyor is a simulation; no camera or belt hardware is connected.